CS6886 – Assignment 2: MobileNet-v2 Compression

Madhav Bhardwaj CS23B035

September 8, 2026

Repository: https://github.com/Madhav1729/cs6886-a2-mobilenetv2-compression

W&B Dashboard: https://wandb.ai/cs23b035-iit-madras/cs6886-a2-mobilenetv2

1 Baseline Training

1.1 Data Pipeline

We used the CIFAR-10 dataset, which has 50,000 training images and 10,000 test images. Table 1 shows how we prepare the images.

Step	Train	Test
1	`RandomCrop(32, padding=4, padding_mode='reflect')`	—
2	`RandomHorizontalFlip(p=0.5)`	—
3	`Resize(160, interpolation=BICUBIC)`	`Resize(160, BICUBIC)`
4	`ToTensor()`	`ToTensor()`
5	`Normalize(ImageNet stats)`	`Normalize(ImageNet stats)`

Table 1: Data preprocessing steps.

We made two main choices here:

- **ImageNet Stats:** The model starts with ImageNet weights, so we keep using ImageNet's normal numbers for color ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). This stops the model from getting confused right away.
- **Resizing to 160×160:** MobileNet-v2 shrinks images a lot as they go through the layers. If we put in a normal 32×32 CIFAR image, the final output size becomes 1×1, which is too small for the model to work with. Resizing to 160×160 leaves a 5×5 size at the end. It also runs much faster than the standard 224×224 size, with almost the same accuracy.

1.2 Model

We use the standard MobileNet-v2 model but change the final layer to output 10 classes instead of 1000. Table 2 shows our settings.

The final layer learns 10 times faster than the rest of the model because it starts from scratch, while the rest of the model already knows how to see images. We also use a one-epoch warmup to keep the training stable at the start.

Parameter	Value
Architecture	Standard `torchvision.models.mobilenet_v2` (2,236,682 params, 8.53 MB)
Pretraining	ImageNet-1K (v1)
Classifier Head	`Linear(1280, 10)`, starting near zero
Optimizer	SGD (momentum 0.9, Nesterov on)
Learning Rate	0.01 (main model), 0.1 (final layer)
LR Schedule	1 epoch warmup, then slowly drop to zero
Weight Decay	4×10^{-5} (except for biases and BatchNorm)
Loss Function	Cross-entropy with 0.1 label smoothing
Epochs / Batch Size	20 epochs, batch size 128

Table 2: Baseline training settings.

Metric	Value
Final Test Accuracy	**96.34%**
Best Test Accuracy (epoch 12)	96.40%
Final Train Accuracy	99.81%
Final Test Loss	0.5916

Table 3: Baseline accuracy results.

1.3 Baseline Results & Errors

The model gets **96.34%** accuracy on the test set (Table 3, Figure 1).

The training accuracy gets very close to 100%, but the test accuracy stops improving around epoch 12. This gap means the model is memorizing the data a bit. It shows the model is bigger than it needs to be for this simple dataset, which makes it a great choice for compression.

Where the Model Fails: As shown in Table 4, the model mostly messes up on things that look alike.

Class	plane	auto	bird	**cat**	deer	**dog**	frog	horse	ship	truck
Accuracy (%)	97.6	98.4	95.9	**91.2**	96.5	**92.7**	98.8	97.4	98.1	96.8

Table 4: Test accuracy for each class.

Almost a quarter of all mistakes happen because it confuses cats and dogs (48 cats guessed as dogs, and 40 dogs guessed as cats). It also confuses trucks and cars 38 times. Since the original images are only 32×32, stretching them to 160×160 doesn't magically add lost details like fur textures. Because of this, the model has a hard time telling apart shapes that are very similar.

2 How Model is compressed

We shrink the model in three steps: removing useless weights (pruning), using smaller numbers (quantization), and packing the data tightly (Huffman coding).

2.1

Pruning: We remove weights that are closest to zero across the whole model at once. This works better than doing it layer by layer. To make sure no single layer gets completely destroyed,

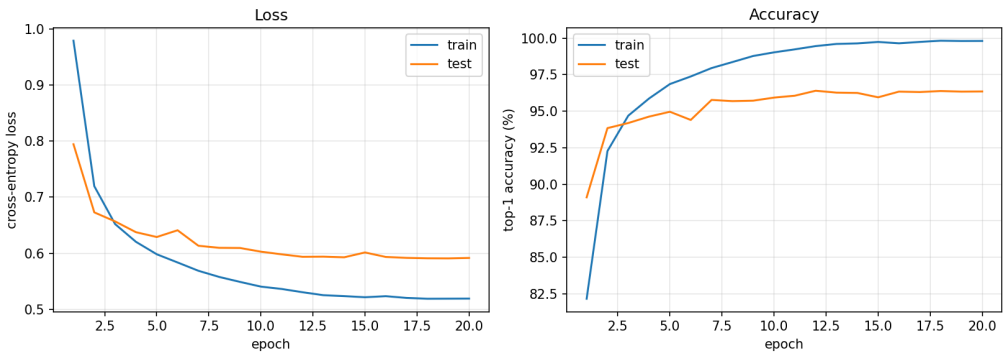

Figure 1: Training progress over 20 epochs.

we cap the removal at 95% per layer. We found that removing 50% of the weights works well. If we try to remove 70%, the model breaks and drops to 13% accuracy.

Weight Quantization: We change the main weights into simple integers (like 4-bit numbers) to save space. We test a few scaling options to find the one that causes the least error, which gives us about a 2% boost in accuracy compared to the simplest method.

Activation Quantization: We also shrink the outputs between the layers (activations) down to 8 bits. Instead of using the absolute maximum and minimum values to set our scale, we ignore the top 0.1% of extreme values. This small change kept the accuracy at 95.85%, whereas the basic method dropped it to 89.55%.

Huffman Coding: Since a lot of the weights become zero after pruning, we use Huffman coding. This is a lossless trick that gives short codes to common numbers (like zero) and longer codes to rare numbers. This saves an extra 15–35% of space.

2.2 Which Layers Matter Most?

MobileNet-v2 is mostly made of two types of layers. The "pointwise" layers hold about 96.5% of the model's parameters. The "depthwise" layers, along with the very first and last layers, hold only about 3.5%.

Layer	Type	Params	Acc. Drop at 3-bit
`features.1.conv.0.0`	depthwise	288	−50.80%
`features.3.conv.1.0`	depthwise	1,296	−2.10%
`features.14.conv.2`	pointwise	92,160	−2.10%
`features.17.conv.2`	pointwise	307,200	−0.25%
`features.18.0`	pointwise	409,600	0.00%

Table 5: How much accuracy drops if we compress just one layer to 3 bits.

As Table 5 shows, if we squeeze a tiny depthwise layer down to 3 bits, the accuracy crashes by 50.80%. But we can squeeze the massive pointwise layers and the accuracy barely moves. Because of this, we use a **mixed strategy**: we compress the big layers a lot (to 3 or 4 bits) and prune them. We leave the small, sensitive layers safely at 8 bits and don't prune them. This trick gave us a huge 14.4% accuracy boost for almost no extra storage cost.

2.3 Storage Details

To report the real size, we have to count everything, including the scales and biases. As Table 6 shows, we keep our scales in 16-bit float (FP16). Trying to squeeze scales down to 8 bits caused too many errors and ruined the model.

Component	Size (KB)	Share
Main weights (Huffman)	1012.5	91.4%
Scales (FP16)	33.3	3.0%
Biases (FP16)	33.3	3.0%
Huffman dictionary	4.9	0.4%
Total	**1049.2**	**100.0%**

Table 6: Size breakdown for the basic 4-bit model without pruning.

3 Testing Different Settings

Before we spent time fine-tuning, we did a quick test to see how different bits and pruning levels behaved.

Main Bits	Output Bits	Sensitive Bits	Accuracy	Space Saved	Size (MB)
8	8	8	96.25%	4.2×	2.018
8	8	4	81.21%	4.3×	1.981
6	8	8	96.04%	5.6×	1.515
6	8	4	77.26%	5.8×	1.478
4	8	8	93.13%	8.1×	1.049
4	8	4	62.64%	8.4×	1.012
3	8	8	40.47%	10.3×	0.832
3	8	4	17.87%	10.7×	0.794

Table 7: Quick test of different bit sizes before fine-tuning.

The main takeaway from Table 7 and Figure 2 is that dropping the sensitive layers from 8 bits to 4 bits always destroys the accuracy. And since those layers are so small, squeezing them barely saves any space anyway. Keeping the sensitive layers at 8 bits is the most important rule we found.

4 Final Results

After picking our best setups, we trained them for 20 more epochs (fine-tuning) to recover the lost accuracy. Table 8 shows the final numbers.

Configuration	Accuracy	Space Saved
4-bit (no pruning)	94.97%	8.13×
Mixed bits (no pruning)	93.74%	9.43×
3-bit (no pruning)	93.10%	9.91×
4-bit + 50% pruning (chosen)	**93.23%**	10.74×

Table 8: Final accuracy after fine-tuning.

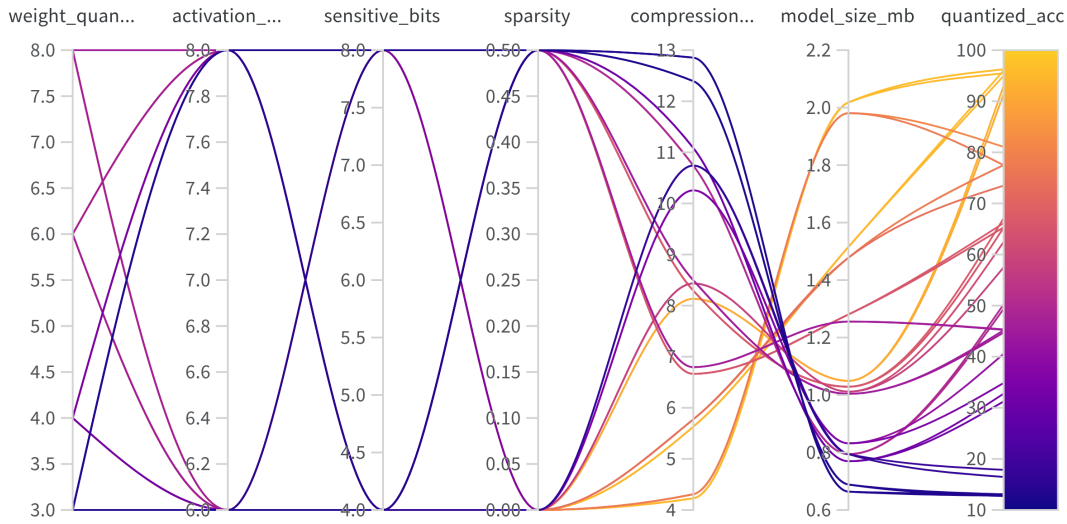

Figure 2: Chart of our 32 test runs. You can see the lines drop down (bad accuracy) whenever we try to compress the sensitive layers to 4 bits.

Our absolute best model uses **50% pruning** on the big layers, combined with **4-bit weights**, **8-bit activations**, and **Huffman coding**:

- **(a) Weight Compression:** The weights take up **11.03**× less space compared to the original model.
- **(b) Activation Compression:** The data moving between layers takes up **4.0**× less space (shrunk from 32-bit floats to 8-bit integers).
- **(c) Accuracy:** It reached **93.23%**. This means we only lost about 3.11% accuracy compared to the original model, which is a great trade-off.
- **(d) Final Size:** The total file size is just **0.795** MB (**813.6** KB), down from 8.53 MB.

Component	**Size (KB)**	**Share**
Main weights (Huffman)	742.5	91.3%
Scales (FP16)	33.3	4.1%
Biases (FP16)	33.3	4.1%
Huffman dictionary	4.5	0.5%
Total Size	**813.6**	**100.0%**

Table 9: Final size breakdown of our best compressed model.